ISHAAN GUPTA

Dallas, TX | (508) 335-5802 | guptaishaan3996@gmail.com
linkedin.com/in/ishaangupta3996 | github.com/Ishaan24687 | ishaan24687.github.io/ishaan-portfolio

Senior Data Engineer | Snowflake · Python · Power BI · Power Automate · Azure

PROFESSIONAL SUMMARY

Senior Data Engineer with 6+ years building production data platforms on Azure and Snowflake. I own the whole path rather than one slice of it: hybrid ingestion across Azure Data Factory, Databricks, and ADLS Gen2, dimensional models in Snowflake with Streams, Tasks, and Snowpark Python, and the Power BI semantic layer that operations teams actually open every morning. Recent work includes migrating a healthcare claims warehouse off SQL Server into a Snowflake star schema now serving 2M+ member records at sub-second latency, and building LokCall, a production call analytics platform at BuzzClan where a single curated employee dimension drives every dashboard, target, and access rule in the product. A lot of what I ship is Python: data quality frameworks, API orchestration, and the ETL that holds the rest together. Microsoft Certified Fabric Data Engineer (DP-700).

CORE COMPETENCIES

- **Snowflake:** Star and snowflake schema design, conformed dimensions and fact tables, SCD Type 2, materialized views, Streams and Tasks, Snowpark Python UDFs, warehouse sizing and query tuning, Time Travel
- **Python Automation:** pandas, PySpark, FastAPI, SQLAlchemy, Pydantic, pytest, dbt, Airflow and APScheduler orchestration, REST API orchestration, declarative data quality frameworks
- **Azure Data Platform:** Data Factory, Synapse, Databricks (Spark, Delta Lake, Unity Catalog), ADLS Gen2, Service Bus, Event Hubs, Azure Functions, Purview, Azure DevOps CI/CD
- **BI and Semantic Modeling:** Power BI Premium, DAX, Power Query, paginated reports, row-level security, Microsoft Fabric, Streamlit, Tableau
- **Workflow Automation:** Power Automate (Teams, SharePoint, Service Bus, vendor API connectors), approval routing, alerting flows, n8n, Zapier
- **Databases and Streaming:** Snowflake, PostgreSQL, SQL Server / SSMS, Oracle, Kafka, Event Hubs, batch and near-real-time
- **Governance and Quality:** Row-level scoping and RBAC, PII redaction (HIPAA), data lineage, drift monitoring, idempotent and reconcilable pipelines
- **Domain:** Healthcare claims (CPT, ICD-10, DRG, EDI 834/837), telephony CDR, utilization and operational reporting

PROFESSIONAL EXPERIENCE

Data Engineer / AI Labs | Lantern Care — Dallas, TX | Oct 2023 - Present

Snowflake Warehouse and Dimensional Modeling

- Claims, eligibility, and member data lived in SQL Server, where month-end reporting locked tables and analysts waited hours for answers. I led the migration to Snowflake and modeled it as a star schema rather than lifting the normalized OLTP tables across: conformed dimensions for member, provider, procedure (CPT), diagnosis (ICD-10), plan and client, facility, and date, with fact tables for claims, appointments, and call activity at their natural grain. Type 2 handling on the member and provider dimensions preserved history the SSMS model had been overwriting. The warehouse holds 2M+ member records, answers analyst queries in under a second, and is the single source every downstream model and dashboard reads from.
- Reloading claims in full every night was getting slow and expensive as volume passed 500K claims a month. I converted the pipeline to incremental loading with Snowflake Streams capturing changes on the landing tables and Tasks running the merge on schedule, plus a reconciliation step comparing row counts and control totals against source before anything publishes. Nightly runtime and warehouse credits both fell, and a bad load now stops at the reconcile gate instead of quietly publishing partial data.
- Processed 500K+ healthcare claims per month across multiple health plans (CPT, ASC, DRG, EDI 834/837), holding 98% data-quality accuracy through schema drift on inbound payer files by validating against the dimension tables rather than trusting the feed.

Azure Pipelines and Lakehouse

- Hinge Health and Sword Health reported member utilization through separate vendor APIs, and some source systems sat on-prem behind the corporate firewall where a cloud-only pipeline could not reach them. I built Yellowstone, a parameterized Azure Data Factory pipeline that pulls both vendors through a Self-Hosted Integration Runtime, lands raw files in ADLS Gen2, and transforms them in Databricks across bronze, silver, and gold layers with Unity Catalog enforcing column-level access and lineage. Because the pipeline is parameterized by vendor, onboarding the second one was configuration rather than a new build, and the same design carries to 10+ vendors.
- Dashboards were serving stale data because the Spark jobs feeding them could not finish inside the refresh window, and the obvious fix of a bigger cluster would have raised the bill. I profiled the jobs, partitioned the Delta tables on the columns queries actually filtered on, Z-ordered the high-cardinality join keys, and right-sized the cluster rather than scaling it up. Query performance improved 60% with no increase in compute spend.

- Care coordinators were reading roughly 30 columns per appointment record by hand to decide who needed outreach, and different coordinators reached different conclusions. I built the HealthHere validation pipeline on Databricks: three decoupled PySpark stages with Delta Lake checkpointing between them, eight business rules held in a config table instead of code, and fuzzy deduplication using Levenshtein distance plus phonetic encoding to collapse near-duplicate members. It runs weekly over 2K+ appointments, eliminated the manual review, and a rule change is now a config edit rather than a deployment.

Power BI, Power Automate and Reporting

- The marketing and growth team was requesting acquisition and outreach campaign numbers over email, and every request became a one-off extract. I built a Power BI semantic model over the Snowflake star schema with DAX measures for campaign reach, engagement, conversion, and cost per acquired member, plus time-intelligence measures so month-over-month and year-to-date comparisons came from one definition instead of five spreadsheets. Row-level security on the client dimension let a single report serve every account without cloning it per client, and marketing self-serves the numbers now.
- Monthly client reporting for 32BJ Anthem, Envision, Hinge Health, 2nd MD, and Dell consumed about four hours of analyst copy-paste per cycle. I built ETL orchestration in Azure Data Factory mapping procedure pricing across bundled, unbundled, and hybrid contracts, then delivered the output as paginated Power BI reports with per-client row-level security. Report generation dropped from four hours to fifteen minutes, and the analyst time went back into the pricing analysis the reports were meant to support.
- Pipeline failures were being discovered when somebody noticed a stale dashboard, which is the worst available detection mechanism. I wired Power Automate flows to pipeline telemetry so a failed load or breached data-quality threshold posts to the owning team's Teams channel with the run ID and failing check, logs the exception to a SharePoint tracking list, and escalates when it goes unacknowledged. The same pattern now routes contract and pricing change approvals, so the request, the approval, and the audit trail live in one flow instead of an email thread.
- Leadership needed to know which metro areas to prioritize for bariatric center-of-excellence contracting, and no single table answered it. I built a Power BI ranking dashboard scoring each MSA on volume, cost variance, and provider availability, and separately delivered IBNR reporting for revenue recognition and a 26-slide claim reimbursement statistical analysis. All three fed contracting and finance decisions rather than sitting in a folder.

Python Automation, Data Quality and MLOps

- Bad source data was reaching dashboards before anyone caught it, and each pipeline carried its own ad-hoc checks. I built a Python data quality framework running declarative rule sets (null thresholds, referential integrity against the dimensions, range and domain checks on CPT and ICD-10 values, row-count drift versus trailing average) as a gate between staging and published tables, writing every result to a Snowflake results table so failures became queryable history instead of a log line. Cataloguing the assets in Azure Purview put lineage and ownership in front of the analysts, not just the engineers.
- Claims arrived from different payers with inconsistent member identifiers, transposed dates of birth, and abbreviated addresses, so adjudication stalled while someone matched them by hand. I built probabilistic record linkage with Splink on the Fellegi-Sunter model: blocking rules cutting candidate pairs by 95%, 10+ comparison features including Jaro-Winkler, Levenshtein, Double Metaphone, and SSN fragments, and a temporal check against the member's coverage window to suppress plausible but impossible matches. I tuned m and u probabilities for precision rather than recall, because a false match costs far more than a missed one in adjudication. Running event-driven off an Azure Storage Queue, it matches in under 3 seconds per claim at 85%+ accuracy and cut adjudication time by 60%.
- Model retrains were ad-hoc notebook runs with no lineage, so when a forecast looked wrong nobody could say which data or code produced it. I packaged training as Docker jobs on Databricks wired to Azure DevOps CI/CD, versioned models in MLflow, and added a drift detector watching population stability index on inputs and the prediction distribution, triggering a retrain when it crosses threshold. Every production model now traces back to its training data and commit, and Power BI dashboards on top surface performance so the team catches degradation before users do.
- Health plan partners were pricing renewals off static actuarial averages that missed cohort-level variance. I built an ensemble on member demographics, prior utilization, CPT mix, and seasonality, choosing XGBoost as the primary over Random Forest because it handled the sparse, high-cardinality CPT features better in cross-validation, and optimized for MAPE rather than RMSE so a handful of catastrophic claims did not dominate the loss. Tuned on $50M+ in annual claims volume to 87% accuracy, it feeds the pricing and contract-renewal workflow directly.
- HealthHere was losing 18 to 20% of scheduled appointments to no-shows, which wasted clinical capacity. I trained a LightGBM classifier on appointment history, engagement signals, lead time, and prior cancellations, then calibrated with Platt scaling so coordinators could read the score as a real likelihood rather than a ranking artifact. Since coordinators can only call so many members a day, I set the decision threshold for precision at their actual outreach capacity instead of maximizing F1. Show-up rates rose 28%.

Applied AI Systems (Production)

- Leadership could not tell where members dropped out of the surgical journey, because the only record was thousands of untranscribed call recordings. I built Auricle on Azure Speech-to-Text and GPT-4o: Azure Functions scaling per call, Container Apps for long-running jobs, and a three-stage extraction pipeline with retry, fallback, and rate limiting, dual-writing to Azure SQL for transactions and Databricks Delta Lake for analytics. The dual write gave analysts 12 dashboard charts of sentiment by journey phase without the transactional system carrying analytical load. It classifies the full IVR volume, roughly 1M calls a month at peak, across 8 surgical journey stages at 94% accuracy with 92% uptime, replacing a manual triage process that had consumed dozens of coordinator hours a week.
- Contract terms were locked inside PDFs, so answering "what is our rate for this CPT with this provider" meant a 30-minute manual lookup. I built Prism: Azure Document Intelligence for OCR and chunking, a seven-step extraction pipeline validated by Pydantic

schemas, Azure AI Search with HNSW indexing for sub-second semantic retrieval across 500+ contracts, and a GPT-4o model fine-tuned on 200+ annotated rate tables. Fine-tuning cut numeric extraction error from about 15% to under 2%, which mattered because a wrong rate is worse than no rate. Lookups now take about 2 seconds with full provenance on every extracted field, and the pipeline handles 100+ contracts an hour.

- Shipped 12 data and AI projects on schedule through Jira sprints, working across clinical, operations, IT, and business teams to turn ambiguous requirements into systems people use daily.

Data Engineer | BuzzClan **Dallas, TX | Jun 2026 - Present**

LokCall, a production PBX call analytics and governance platform (PostgreSQL, Python, FastAPI, Django, Google Cloud)

- Call analytics were keyed to raw PBX extensions, so every time someone changed teams or picked up a new handset their history broke and managers lost the trail. I rebuilt the model around a proper employee dimension: a CSV-driven `dim_employee` loaded by an idempotent Python ETL, surrogate keys on every fact row, and a `vw_call_employee_details` view joining call records to the current org structure. Re-keying targets and dashboards to directory identity instead of extension meant history survived reorgs, and one directory upload now propagates team structure to every dashboard, target, and permission in the product.
- Audio recordings landed in Google Drive while call detail records arrived separately from the PBX, and the two sources disagreed without anyone noticing. I built a source-aware ingestion pipeline that parses the encoded filename into date, extension, phone number, and call ID, reconciles it against the CDR export, and writes idempotently so a re-run never double-counts. A Pipeline Overview screen now shows coverage per source, so the team can see which calls are missing audio versus missing metadata instead of guessing.
- Auto-transcription ran once nightly and could never catch up, leaving recent calls untranscribed for days. I moved it to a continuously draining worker that uploads audio to Google Cloud Storage, submits Speech-to-Text V2 batch jobs (roughly 75% cheaper than the real-time API), polls the long-running operation, and cleans up the bucket afterward. The backlog cleared and transcription now keeps pace with uploads rather than falling behind them.
- Large directory and CDR uploads were returning 502s because a heavy analytics aggregation ran inline on the web worker. I traced it across all four upload and reconcile entry points, not just the one that had been reported, and moved the refresh to a background task. Uploads return immediately now, and the fix held for every path rather than resurfacing on the next one.
- Managers needed to see their own teams without seeing everyone, which is the same problem as row-level security in a BI tool. I implemented a three-scope model (corporate, team, self) enforced as a FastAPI dependency and a Django decorator on every view, backed by a shared visibility service that rewrites the WHERE clause per scope so no route can leak by accident. I also cleared the automated security gate's SQL-injection findings by parameterizing the analytics queries and removed a hardcoded secret it flagged as critical.
- Promoting LokCall to production meant moving schema and curated reference data with no maintenance window. I wrote parity tooling that reconciles column schemas, refreshes stale views, and migrates call records, transcripts, and IAM rows through idempotent Alembic migrations with a raw-SQL fallback for the re-key step. Cutover ran clean, across 71 commits and 15 pull requests that each passed an automated static-analysis and code-review gate.

Contract Programmer Analyst | City of Red Bank, TN **Remote | Dec 2025 - Mar 2026**

A full-stack municipal tax simulation and analytics platform (Python, SQL, interactive dashboard)

- City council was weighing millage-rate changes with no way to see the effect on residents or how Red Bank compared to peer cities, so every rate discussion ran on guesswork. I built a full-stack tax simulation platform that models millage-rate scenarios in real time, benchmarks the city against comparable Tennessee municipalities, and surfaces property-tax burden by scenario, so staff and council could test a rate change and see who it touched before voting on it.
- The inputs lived in disconnected sources — assessment rolls, exemption records, and peer-city comparables — in formats nothing could join. I developed the back-end data pipelines in Python and authored the SQL to ingest and reconcile all three into tidy, analysis-ready datasets feeding the simulation engine, so the model ran on one clean source instead of a pile of spreadsheets.
- Council members are not analysts, so the numbers had to be explorable without a spreadsheet. I built the interactive dashboard — scenario sliders, peer-city benchmarks, and ward-level tax-burden drill-downs — that city staff and council used to weigh rate changes against revenue trade-offs directly.
- A tool nobody can run gets abandoned when the contract ends. I delivered classroom-style training and walk-through sessions for city staff on operating the platform, interpreting its outputs, and refreshing the underlying data, so the city could keep it running without me.

Data Analyst Intern | Tech Mahindra **Plano, TX | Jun 2023 - Sep 2023**

- Automated reporting jobs were running high-frequency Oracle queries that had accumulated redundant joins over years of edits, and the compute bill showed it. I rewrote the worst offenders, removed the duplicate joins, and added indexed lookups on the columns the reports actually filtered on. Execution time fell 25%, worth roughly $30K annualized in cloud compute.
- Marketing was running email and SMS campaigns through Twilio and SendGrid with no reliable view of what was landing. I owned those delivery pipelines and instrumented them end to end, feeding bounces, opens, and clicks back into the reporting layer. Delivery reached 98% and outbound engagement rose 40%.

- Campaign performance was only ever reviewed after the fact, from exported CSVs. I built Power BI dashboards over campaign and customer behavior data so marketing could watch performance while a campaign was still running, which turned the conversation from post-mortem into mid-flight adjustment.

Data Analyst | Johnson Matthey **Gurugram, India | Dec 2018 - Jun 2021**

- A specialty product line was carrying overstock and stockouts at the same time, which usually means the forecast is wrong at SKU level even when it looks fine in aggregate. I built regression and ensemble models on historical demand, seasonality, and lead time, and tuned at SKU granularity rather than product family. Both problems eased, contributing roughly $50K in margin improvement on that line.
- Data prep for reporting was manual and inconsistent between analysts, so two people could produce two different numbers from the same source. I designed ETL pipelines in Azure Data Factory and Apache NiFi to standardize ingestion and cleansing, cutting processing effort 20% and removing the ambiguity.
- Engineering and manufacturing bills of materials were reconciled by hand, so changes propagated slowly and occasionally not at all. I automated EBOM to MBOM coordination in Python, improving manufacturing efficiency 15% across the linked workflows, and built Tableau and Power BI dashboards on the KPIs leadership tracked. That visibility supported a 15% reduction in customer churn.

EDUCATION

M.S. in Computer Science (STEM) | Clark University, Worcester, MA | GPA 3.94 / 4.0 | Jan 2022 - Aug 2023

B.Tech in Engineering | Guru Gobind Singh Indraprastha University, Delhi, India | May 2014 - May 2018

CERTIFICATIONS

- **Microsoft Certified: Fabric Data Engineer Associate (DP-700)**
- Azure Machine Learning
- Databricks Foundations
- Data Engineering Specialization
- Complete Machine Learning and Data Science
- SQL Masterclass
- Google Analytics (Advanced)

TECHNICAL SKILLS

- **Data Warehousing:** Snowflake (Snowpark, Streams, Tasks, materialized views, Time Travel), star and snowflake schema, SCD Type 2, medallion architecture, dbt
- **Cloud and Pipelines:** Azure Data Factory, Synapse, Databricks (Spark, Delta Lake, Unity Catalog), ADLS Gen2, Azure Functions, Service Bus, Event Hubs, Apache NiFi, SSIS, Fivetran, Kafka
- **Programming:** Python (Expert), SQL (Advanced), T-SQL, PL/SQL, R; pandas, NumPy, PySpark, FastAPI, SQLAlchemy, Pydantic, pytest, Dask
- **BI and Reporting:** Power BI Premium, DAX, Power Query, paginated reports, row-level security, Microsoft Fabric, Power Automate, Tableau, SSRS, Streamlit, Plotly, Qlik Sense
- **Databases:** Snowflake, PostgreSQL, SQL Server / SSMS, Oracle
- **MLOps and DevOps:** Azure ML Studio, Azure AI Foundry, Azure OpenAI, MLflow, Docker, Azure DevOps, GitHub Actions, Jenkins, Bitbucket Pipelines, Alembic
- **ML:** scikit-learn, XGBoost, LightGBM, CatBoost, PyTorch, TensorFlow, Splink, ensemble methods
- **GenAI:** GPT-4 / GPT-4o, Claude, Gemini, RAG, LangChain, LangGraph, MCP tool integrations, prompt engineering, fine-tuning
- **Automation:** Power Automate, n8n, Zapier; AI coding tools: Claude Code, Cursor, GitHub Copilot, Windsurf